Conversación con Claude -- Javier Ovelar

Claude Code (Opus 5.5) · Diagnóstico Fase 1: cambio de estado y asignación de responsable · 24/10/2026

> **Usuario (Javier Ovelar) 24/10/2026**
>
> Hola, siendo javier, necesito que me guies en el proceso de desarrollo, para este caso quiero realizar el diagnostico sobre la parte que me fue asignada

Claude 24/10/2026 11:34

Voy a revisar el repositorio para encontrar qué parte te fue asignada y cómo hizo Cristian su diagnóstico.

Ran 2 shell commands

Ahora veo main.py para ubicar las partes restantes, y busco el reparto de tareas.

Ran 1 shell command

No encuentro en el repo qué parte le toca a cada uno. Busco el reparto en el PDF de preparación de Cristian.

Ran 3 shell commands

Ya tengo tu asignación. Leo el plan (checklist individual) y el resto del código que necesitas para tu parte.

Ran 1 shell command

Antes de proponerte casos de prueba, verifico qué responde de verdad el sistema. Lo corro sobre una copia en el scratchpad para no tocar tu base.

Ran 1 shell command

Confirmo un detalle del historial (la reasignación del caso A7):

Ran 1 shell command

Javier, ya tengo todo lo necesario. Según el plan de trabajo (en el PDF de preparación de Cristian), tu parte es el cambio de estado y la asignación de responsable, o sea las líneas 251–318 de main.py:

- proc_estado → PUT /solicitudes/{id}/estado (251–288)
- asignar → PUT /solicitudes/{id}/responsable (291–318)

En el plan también figura que coordinás el code review y /docs, que en la Fase 3 te toca garantizar el Move Method y que Gaston revisa tu PR.

Ya estás en la rama docs/diagnostico-javier, creada después del merge del PR #3, así que podés arrancar. Para que el entregable sea parejo con el del grupo, conviene seguir el mismo formato que Cristian: dos documentos más las capturas.

Paso 1 — Leer tu código

Mientras lo leés, fijate en estos puntos (los vas a usar en la ficha):

- 260–265: las transiciones de estado están armadas con una cadena de if.
- 267, 269, 271, 296, 298, 302, 304, 306: los errores se devuelven con return {"error": ...}, que responde 200. En cambio, la línea 256 usa HTTPException(404). Hay dos criterios distintos dentro de tu propia parte.
- 270: el motivo de rechazo solo se compara con "".

- 279–286: la notificación tiene un except: pass sin tipo y es casi igual a la de registrar (190–194, de Gaston).
- 309–311: asignar cambia el estado por su cuenta, por fuera de las reglas de proc_estado.
- 314–316: el historial solo se escribe si cambia el estado.
- Índices sueltos como s[5], s[6], s[7], r[2] y r[4], y el nombre id, que tapa una función de Python.

Paso 2 — Calidad externa (pruebas manuales)

1. Ejecutá .\run.bat reset y levantá el sistema con .\run.bat.
2. Abrí http://127.0.0.1:8000/docs.
3. Ejecutá los casos en este orden, porque cada uno cambia los datos para el siguiente.
4. Guardá las capturas en docs/evidencias/fase1/javier/ (por ejemplo, caso01_estado_no_existe.png).

Los corrí antes en una copia aislada, así que esto es lo que deberías ver. Si te aparece otra cosa, anotalo, porque también es un hallazgo.

#	Endpoint	id	Body	Debería responder
1	estado	999	{"estado":"ASIG"}	404 {"detail":"no existe"}
2	responsable	999	{"responsable_id":1}	200 {"error":"solicitud no existe"} ← comparalo con el caso 1
3	estado	1	{}	200 "transicion no permitida de PEND a None"
4	estado	1	{"estado":"ASIG"}	200 "no tiene responsable"
5	estado	1	{"estado":"RES"}	200 "transicion no permitida..."
6	responsable	1	{}	200 "responsable no existe" (el mensaje engaña: en realidad falta el campo)
7	responsable	1	{"responsable_id":99}	200 "responsable no existe"
8	responsable	1	{"responsable_id":2}	200 "responsable inactivo"
9	responsable	1	{"responsable_id":4}	200 "no pertenece al area Alumnado"
10	responsable	1	{"responsable_id":1}	200, pasa a ASIG ✓
11	responsable	4	{"responsable_id":5}	200 "la solicitud esta cerrada"
12a	POST /responsables	—	{"nombre":"Laura Gil","area":"Departamento Academico","email":"lgil@ejemplo.com"}	crea el responsable con id 6
12b	responsable	2	{"responsable_id":6}	200, reasigna
12c	GET /solicitudes/2	—	—	el historial no registra la reasignación (sigue diciendo "asignada a Paula Diaz")
13	estado	2	{"estado":"EN_PROC"}	200 ✓; mirá notificaciones.log
14	estado	2	{"estado":"RECH"}	200 "para rechazar hay que poner motivo"
15	estado	2	{"estado":"RECH","comentario":null}	200, rechaza sin motivo ← defecto
16	estado	3	{"estado":"RECH","comentario":" "}	200, rechaza con un motivo vacío
17	GET /solicitudes/2 y /3	—	—	historial con comentario null y " "
18	estado	1	{"estado":"CANC","comentario":123}	200, guarda "123" (no valida el tipo)

19	estado	6	[1, 2]	422 con formato detail (igual que el caso 12 de Cristian)

Usá el mismo criterio de códigos HTTP que Cristian (201 / 422 / 409 / 404 / 500), así los resultados se pueden comparar.

Paso 3 — Ficha de smells

Estos son los hallazgos que veo como candidatos. El impacto, el riesgo, la prioridad y el principio de cada uno redactalos vos, como hizo Cristian: es la parte que se evalúa y que te van a preguntar.

Candidato	Líneas	Casos
Manejo de errores inconsistente (200 / 404 / 422)	256, 267–271, 296–306	1–9, 11, 14, 19
Switch Statements / reglas de transición hardcodeadas	260–265	3, 5
Lógica de estados repartida (asignar cambia el estado por su cuenta; Shotgun Surgery)	309–311 vs. 260–265	4, 10
Validación débil del motivo de rechazo	270	15, 16
Falta de modelo de entrada (data: dict)	252, 292	3, 6, 18, 19
Pérdida de trazabilidad en reasignaciones	314–316	12b, 12c
except: sin tipo que silencia errores	285	— (desde el código)
Duplicated Code: notificación, insert de historial, formato de fecha	273–286, 308, 315 (y 180–194)	13
Magic Numbers (índices de columnas)	258, 268, 297, 303, 305	—
Magic Strings / Primitive Obsession (estados como texto)	260–265, 297	—
Long Method / SRP en proc_estado	251–288	—
Nombres poco claros (proc_estado, s, r, act, id)	varias	—

Un detalle para la ficha: como asignar siempre pasa de PEND a ASIG, el camino PEND → ASIG por /estado nunca puede tener éxito usando la API (caso 4).

Puntos para coordinar con el grupo:

- Gaston: la notificación y el insert de historial están duplicados con registrar.
- Bruno: la lista ["RES","RECH","CANC"] se repite en reporte (línea 346).

Paso 4 — Archivos a crear

- docs/diagnostico/javier_resultados_fase1.md
- docs/diagnostico/javier_ficha_smells.md
- docs/evidencias/fase1/javier/*.png
- docs/uso-ia/javier.md, y esta conversación exportada a anexo/ia/javier_claude_01_diagnostico.pdf

Paso 5 — Commit y PR

- Commit: docs: diagnóstico de cambio de estado y asignación
- PR hacia dev usando la plantilla. Marcá "docs", etapa "diagnóstico", y pedile la revisión a Gaston.

Si querés, te armo los dos .md con la misma estructura que los de Cristian: la tabla de casos con lo que respondió el sistema, y la ficha con ubicación, evidencia y casos ya cargados, dejándote vacíos los campos de análisis. ¿Lo hago, o preferís ejecutar primero los casos y pasarme lo que obtuviste?

> **Usuario (Javier Ovelar) 24/10/2026**
>
> ok

Claude 24/10/2026 11:36

[Write] docs\diagnostico\javier_resultados_fase1.md — Wrote 75 lines to docs\diagnostico\javier_resultados_fase1.md

[Write] docs\diagnostico\javier_ficha_smells.md — Wrote 159 lines to docs\diagnostico\javier_ficha_smells.md

Javier, ya te armé los dos documentos con la misma estructura que los de Cristian:

- docs/diagnostico/javier_resultados_fase1.md: tiene los 19 casos en el orden de ejecución, los datos iniciales que necesitás para seguirlos y los nombres de las capturas.
- docs/diagnostico/javier_ficha_smells.md: tiene los 12 hallazgos con la ubicación, la evidencia y los casos que los respaldan, más las notas para coordinar con Gaston, Bruno y Cristian.

Lo que te queda a vos:

1. Ejecutar los casos. Corré .\run.bat reset y .\run.bat, y hacé los casos en orden desde /docs. Guardá cada captura en docs/evidencias/fase1/javier/ con el nombre que figura en la tabla.
2. Completar la tabla de resultados. Las columnas HTTP y "Qué respondió" tienen lo que obtuve yo en una corrida de prueba: confirmalas o corregilas con lo que te dé a vos. Después completá "¿Está bien?", "¿Por qué?", la fecha y las conclusiones.
3. Completar la ficha. Faltan el impacto, el riesgo, la prioridad y el principio de cada hallazgo, el criterio de priorización y tu propuesta para la tabla grupal. Esto conviene que lo escribas vos, porque es lo que se evalúa.
4. Registrar el uso de IA. Creá docs/uso-ia/javier.md (te puedo armar la base) y exportá esta conversación a anexo/ia/javier_claude_01_diagnostico.pdf.
5. Hacer el commit y el PR. Cuando termines, avisame y te ayudo a abrir el PR hacia dev, con Gaston como revisor.

Si en algún caso te sale algo distinto de lo que está en la tabla, pasame la captura y lo vemos juntos.

> **Usuario (Javier Ovelar) 24/10/2026**
>
> Tengo una duda, en el test 8 devuelve 200, encontro el responsable pero esta inactivo. se puede considerar el codigo valido?

Claude 24/10/2026 15:35

No, el 200 no es válido. La lógica sí está bien: detecta que el responsable está inactivo, no lo asigna y el mensaje es claro. El problema es el código HTTP. Un 200 le dice al cliente que la operación salió bien, y en realidad la asignación no se hizo. Si un frontend mira solo el código, va a creer que la solicitud quedó asignada.

¿Qué código correspondería?

- 404 no: el responsable existe, así que no es "no encontrado".
- 409 Conflict: es el que te recomiendo. El pedido es correcto en su forma, pero choca con el estado actual de un recurso: el responsable está inactivo. Coincide con la fila "Choca con el estado actual del recurso" de tu tabla de criterios.

- 422 también se puede defender, si lo pensás como "el responsable_id enviado no es válido para esta operación".

Elijas cuál elijas, usalo igual en los casos parecidos:

- Caso 9 (responsable de otra área)
- Caso 11 (solicitud cerrada)
- Caso 4 (pasar a ASIG sin responsable)

Cómo cargarlo en la tabla: igual que Cristian en sus casos 2 y 3.

- ¿Está bien? Parcial
- ¿Por qué? Detecta que el responsable está inactivo y no lo asigna, pero responde 200 como si la operación hubiera salido bien; debería responder 409 Conflict.

Este caso es evidencia del hallazgo 1 de tu ficha (manejo de errores inconsistente), que ya lo tiene listado.

> **Usuario (Javier Ovelar) 24/10/2026**
>
> En el caso 14, el comentario se envio como campo nulo (null), pero devolvio 200 y acepto la solicitud. Puede ser que haya fallado alguna validacion?

Claude 24/10/2026 16:48

Reviso cómo quedó el caso 14 en tu archivo, porque en la tabla original el envío con null era el 15.

Searched for 1 pattern

Ojo con la numeración: el envío con "comentario": null es el caso 15. El 14 es el que no manda el campo comentario, y ese ya lo completaste bien.

Lo que viste en el 15 es justamente el defecto que buscaba ese caso. No es que la validación se haya roto: está incompleta. Es la línea 270 de main.py:

```
if nuevo == "RECH" and ("comentario" not in data or data["comentario"] == ""):
    return {"error": "para rechazar hay que poner motivo"}
```

Con {"estado":"RECH","comentario":null} pasa esto:

1. "comentario" not in data da False, porque la clave existe aunque su valor sea null.
2. data["comentario"] == "" da False, porque None no es igual a "".
3. Como las dos condiciones son falsas, no entra al if y sigue de largo.
4. Rechaza la solicitud y guarda el motivo como null en el historial (lo ves en el caso 17).

La validación solo contempla dos situaciones: que falte el campo o que venga vacío. No contempla null, ni un texto en blanco (caso 16), ni otro tipo de dato (caso 18).

Cómo cargarlo en la tabla:

- ¿Está bien? No
- ¿Por qué? Rechaza la solicitud sin motivo, cuando el motivo es obligatorio para rechazar. La validación solo controla que el campo falte o sea "", no que sea null (línea 270). Debería responder 422 sin cambiar el estado.

A diferencia del 14, este no es "Parcial": en el 14 el sistema frenó la operación y solo falló el código HTTP, y acá dejó pasar un dato inválido y lo guardó. Es evidencia del hallazgo 4 de tu ficha.

⚠️ La solicitud 2 quedó en RECH, que es un estado final, así que no se puede repetir el caso sobre ella. Si necesitás rehacer la captura, tendrías que correr .\run.bat reset, que borra todos los datos actuales, y volver a ejecutar desde el caso 1, porque los casos dependen del orden. Si ya tenés la captura, no hace falta.

> **Usuario (Javier Ovelar) 24/10/2026**
>
> quise decir el caso 15

Claude 24/10/2026 16:49

Perfecto, entonces lo que te expliqué aplica tal cual: hablábamos del 15.

En resumen, no falló ninguna validación. La de la línea 270 está incompleta: solo controla que el campo falte o sea "", y como null no cumple ninguna de las dos condiciones, el rechazo pasa sin motivo. En la tabla va "No", con la justificación que te pasé.

> **Usuario (Javier Ovelar) 24/10/2026**
>
> Creo que termine con las pruebas del 1 al 19, que sigue?

Claude 24/10/2026 17:43

Reviso cómo quedaron la tabla y las capturas antes de pasar al siguiente paso.

Ran 1 shell command

Las 19 pruebas están completas y tenés todas las capturas. Antes de pasar a la ficha, conviene cerrar este documento. Te dejo lo que encontré al revisarlo, de lo más importante a lo menos.

1. Correcciones en la tabla

Caso 6: pusiste que debería ser 404, pero ahí falta un campo. Según tu propio criterio, eso es 422, igual que en el caso 3. Además, en el 7 ya usás 404 para "el responsable no existe"; si el 6 también fuera 404, no se distinguirían. Por ejemplo:

> Código incorrecto, debería ser 422 porque falta responsable_id. El mensaje es engañoso: dice que el responsable no existe, cuando en realidad no se envió.

Caso 12c: no es un problema del mensaje. El GET funciona bien; lo que falla es que la reasignación del 12b no quedó registrada. Por ejemplo:

> El GET funciona, pero el historial no registra la reasignación a Laura Gil: no queda constancia de quién tenía la solicitud antes ni de cuándo cambió. Falla la trazabilidad.

Caso 15: "falló validación o es débil" queda ambiguo. Ahora ya sabés la causa, así que ponela:

> Rechaza la solicitud sin motivo. La validación de la línea 270 solo controla que el campo falte o sea "", no que sea null. Debería responder 422 sin cambiar el estado.

Caso 1: el 404 está bien, pero la justificación habla del mensaje. Por ejemplo:

> El código 404 es correcto. El formato (detail) no coincide con el del resto de los errores (error).

Caso 19: sumale lo mismo, que el formato detail es distinto del de los otros errores.

Caso 18: dice "Sting", va "String" (o "texto").

2. Capturas con doble extensión

Cinco archivos terminan en .png.png, así que no coinciden con los nombres de la tabla y los links no van a andar:

- caso01_estado_no_existe.png.png
- caso02_asignar_no_existe.png.png
- caso13_notificaciones_log.png.png
- caso18_comentario_numero.png.png
- caso19_body_lista_422.png.png

Si querés, los renombro yo. Solo se cambia el nombre, no se borra nada.

3. Lo que falta completar

- Fecha de ejecución.
- Conclusiones, en 4 o 5 puntos como las de Cristian:
  - Funcionalidad básica: las asignaciones y transiciones válidas funcionan (casos 10 y 13).
  - Contrato HTTP: casi todos los errores responden 200, y la misma situación responde 404 en un endpoint y 200 en el otro (casos 1 y 2).
  - Validación: se aceptan motivos nulos, en blanco o numéricos (casos 15, 16 y 18), y faltan campos sin que se avise bien (casos 3 y 6).
  - Trazabilidad: las reasignaciones no quedan en el historial (casos 12b y 12c).
  - Defectos comprobados: los casos 12c, 15 y 16 no son riesgos leídos en el código, sino fallas reproducibles.

Escribilas vos con tus palabras; después las reviso si querés.

Lo que sigue

1. Ficha de smells (javier_ficha_smells.md): es el trabajo más importante. Para cada hallazgo completás el impacto, el riesgo, la prioridad y el principio; podés guiarte por cómo lo hizo Cristian. Arrancá por los que tienen casos (1, 3, 4, 5 y 6), porque ya tenés la evidencia fresca.
2. Registro de uso de IA (docs/uso-ia/javier.md) y exportar esta conversación a PDF.
3. Commit y PR hacia dev, con Gaston como revisor.

¿Renombro las capturas?

> **Usuario (Javier Ovelar) 24/10/2026**
>
> ahora corrijo yo las inconsistencias

Claude 24/10/2026 17:45

Dale. Cuando termines, avisame y lo reviso.

Mientras corregís, acordate de dos cosas:

- Las capturas .png.png: si las renombrás a mano, fijate que el nombre quede exactamente igual al de la columna Evidencia.
- El caso 6 con 422: si lo cambiás, revisá que el resto de la tabla siga el mismo criterio. Un campo que falta es 422 y un recurso que no existe es 404.

Después seguimos con la ficha de smells.

> **Usuario (Javier Ovelar) 24/10/2026**
>
> Listo, sigamos con el informe

Claude 24/10/2026 17:59

Primero reviso las correcciones y después seguimos con la ficha.

Ran 1 shell command

Las capturas ya coinciden con la tabla, y también corregiste la fecha y los casos 6 y 18.

Todavía falta en los resultados:

- Casos 1, 12c, 15 y 19: las justificaciones siguen como estaban. Si decidiste dejarlas así está bien, pero el 15 te conviene corregirlo: "falló validación o es débil" no dice cuál es la causa, y ahora la sabés.
- Conclusiones: siguen vacías. Las podés escribir al final, después de la ficha.

Ficha de smells

En cada hallazgo ya están cargados la ubicación, la evidencia y los casos. Te faltan cuatro campos, y cada uno responde una pregunta:

Campo	Pregunta que responde	Ejemplo de Cristian
Impacto	¿Qué problema causa hoy, a quien usa el código o lo mantiene?	"el cliente recibe respuestas de error con estados HTTP incorrectos"
Riesgo	¿Qué puede salir mal en el futuro si no se corrige?	"el frontend puede interpretar un fallo como una operación exitosa"
Prioridad	Alta / Media / Baja, y por qué	"alta, porque hay defectos comprobados"
Principio	¿Qué principio viola?	SRP, OCP, DRY, DIP, Clean Code: nombres, manejo de errores, fail fast...

Criterio de prioridad: te recomiendo usar el mismo que Cristian, así la tabla grupal queda pareja.

- Alta: el hallazgo tiene un defecto comprobado con un caso, o bloquea los tests.
- Media: dificulta el mantenimiento, pero no hay una falla comprobada.
- Baja: es sobre todo un problema de legibilidad.

Con ese criterio, a modo orientativo: 1, 4, 5 y 6 serían altas (todos tienen casos con fallas); 2, 3, 7, 8 y 10, medias; 9, 11 y 12, entre media y baja. La decisión final es tuya, pero cada prioridad tiene que tener su "porque".

Para los principios, algunas pistas:

- 1: manejo de errores.
- 2 y 3: OCP. Si mañana hay un estado nuevo, ¿cuántos lugares hay que tocar?
- 4 y 5: validación explícita / fail fast.
- 6: pensá qué regla de negocio se pierde. Todo cambio de una solicitud debería quedar registrado.
- 7: manejo de errores. ¿Qué pasa si la notificación falla y nadie se entera?
- 8: DRY.
- 11: SRP.

Cómo seguir: escribí primero el hallazgo 1 completo y pasámelo. Lo reviso, ajustamos el nivel de detalle y con eso de modelo hacés el resto más rápido.

> **Usuario (Javier Ovelar) 24/10/2026**
>
> quise decir, mejor las conclusiones de la tabla

Claude 24/10/2026 17:59

Dale, hacemos las conclusiones. Te armo la estructura con los datos de tu tabla y la redacción la hacés vos.

Datos de tu tabla

- 21 filas: 5 "Sí" (1, 10, 12a, 12b y 13), 10 "Parcial" (2, 4, 5, 7, 8, 9, 11, 12c, 14 y 19) y 6 "No" (3, 6, 15, 16, 17 y 18).
- 10 errores responden 200: casos 2, 3, 4, 5, 6, 7, 8, 9, 11 y 14.
- El único 404 es el caso 1, y el único 422 es el caso 19, que lo genera FastAPI y no tu código.

Estructura (mismo formato que Cristian)

Cada punto es un título en negrita más una o dos oraciones con los casos entre paréntesis.

Funcionalidad básica ¿Las operaciones válidas funcionan? Mirá los casos 10, 12b y 13: asignación, reasignación, transición y notificación.

Contrato HTTP inconsistente

- ¿Con qué código responden los errores? (los 10 casos de arriba)
- La comparación más fuerte que tenés es la de los casos 1 y 2: la misma situación, "la solicitud no existe", responde 404 en un endpoint y 200 en el otro.
- Hay dos formatos de error: detail (casos 1 y 19) y error (el resto).
- La conclusión que se desprende: ¿puede el cliente saber si la operación salió bien mirando solo el código?

Validación incompleta

- Los campos obligatorios que faltan no se informan bien: los mensajes son engañosos (casos 3 y 6).
- El motivo de rechazo acepta null y texto en blanco (casos 15 y 16), y el comentario acepta números (caso 18).

Trazabilidad ¿Qué pasa con el historial cuando se reasigna? (casos 12b y 12c) ¿Y qué queda guardado cuando se rechaza sin motivo? (caso 17)

Defectos comprobados ¿Cuáles de estos casos no son riesgos leídos en el código, sino fallas que reprodujiste? Pensá en el 12c, el 15 y el 16: el sistema dejó guardados datos incorrectos o incompletos.

Opcional, si querés sumar algo propio: en el caso 4, como asignar siempre pasa la solicitud de PEND a ASIG, la transición PEND → ASIG por /estado nunca puede tener éxito. Es una observación que no salió en el diagnóstico de Cristian.

Escribilas en el archivo, reemplazando el comentario <!-- ... -->, y avisame para revisarlas.

> **Usuario (Javier Ovelar) 24/10/2026**
>
> hay complete lo que indicabas, que mas faltaria para completar la etapa 1?
>
> *Ran 1 shell command*

Claude 24/10/2026 18:34

Tu parte de la Etapa 1 está casi lista. La ficha quedó bien en general: cada hallazgo tiene sus cuatro campos completos y la prioridad justificada. Antes de cerrarla, te marco tres ajustes y después lo que falta para terminar la etapa.

Ajustes en la ficha

1. El orden de resolución contradice tus prioridades. Ahora los hallazgos 4, 5 y 6 (prioridad alta, con defectos comprobados) quedan al final, después del 2 y el 10, que son de prioridad media. Te propongo invertirlo:

> Primero el 1, porque es transversal (es el mismo problema que el hallazgo 3 de Cristian); después 4, 5 y 6, que tienen defectos comprobados; por último 2, 3 y 10 juntos, moviendo las transiciones a un módulo de estados (Move Method).

Si preferís mantener tu orden, agregá por qué conviene hacer primero el Move Method.

2. Hallazgo 3: la prioridad dice "comprobado en los casos 4 y 10", pero en tu tabla el 10 es "Sí", así que funcionó bien. El defecto se comprueba solo con el 4. El 10 sirve como evidencia de que asignar cambia el estado por su cuenta, pero no como falla. Además, para este hallazgo SRP no encaja del todo: el problema es que la misma regla vive en dos lugares, así que DRY encaja mejor (una sola fuente de verdad para las transiciones).
3. "Asignar no notifica al estudiante" aparece en la conclusión 4 y en el impacto del hallazgo 3, pero no tenés una captura que lo muestre. En el caso 10 solo capturaste la respuesta. Tenés dos opciones:
4. abrir ahora notificaciones.log, comprobar que no hay ninguna línea de la asignación de la solicitud 1, capturarlo y agregarlo en la fila 10 (caso10_notificaciones_log.png), o
5. sacar esa afirmación.

Para esa captura no hace falta repetir nada ni resetear la base: basta con abrir el archivo.

En los resultados, las filas 1, 12c, 15 y 19 siguen como estaban. Si lo decidiste así, está bien.

Lo que falta para cerrar la Etapa 1

Tuyo:

1. docs/uso-ia/javier.md: la declaración de uso de IA, con el mismo formato que la de Cristian. Ahí va qué hizo la IA (identificar tu parte, proponer los casos, armar las plantillas, revisar) y qué hiciste vos (ejecutar y capturar los 19 casos, evaluarlos y redactar el análisis de la ficha y las conclusiones).
2. Exportar esta conversación como anexo/ia/javier_claude_01_diagnostico.pdf. Con el comando /export de Claude Code la guardás como texto, y después la pasás a PDF.
3. Commit y PR hacia dev: commit docs: diagnóstico de cambio de estado y asignación, la plantilla completa (tipo "docs", etapa "diagnóstico", sección de uso de IA) y Gaston como revisor.

Del grupo, después de que se mergee tu PR:

4. La puesta en común: armar la tabla grupal de smells (la coordina Gaston) con tu propuesta y las coordinaciones que anotaste.
5. El tag etapa-diagnostico sobre dev, cuando estén los cuatro diagnósticos (lo hace Cristian).

Para más adelante: como coordinás el code review, en el plan figura que podés ir proponiendo el borrador del checklist de code review.

¿Te armo la base de docs/uso-ia/javier.md con los campos de análisis para que los completes vos?

> **Usuario (Javier Ovelar) 24/10/2026**

> como se usa el comando /export?

Claude 24/10/2026 18:36

/export es un comando de Claude Code que se escribe en el mismo lugar donde me escribís los mensajes. Hay dos formas de usarlo.

1. Sin argumentos

```
/export
```

Se abre un menú para elegir entre copiar la conversación al portapapeles o guardarla en un archivo.

2. Con un nombre de archivo

```
/export javier_claude_01_diagnostico.txt
```

La guarda directamente en ese archivo, dentro de la carpeta del proyecto.

Recomendaciones

- Exportá al final, cuando terminemos todo lo del diagnóstico (incluido el PR). Así queda la conversación completa. Si la exportás ahora y seguimos hablando, después tendrías que volver a hacerlo.
- No subas el .txt al repo. Se guarda en la raíz del proyecto, así que al hacer el commit fijate de no agregarlo. Lo que va al repo es el PDF, en anexo/ia/. Cuando tengas el PDF, podés borrar el .txt o moverlo fuera del proyecto.
- Revisalo antes de convertirlo. El texto incluye también las salidas de los comandos que ejecuté (lecturas de archivos y la simulación de los casos). Si querés que el anexo quede más limpio, preguntale a Cristian si recortó algo del suyo.

Pasarlo a PDF

1. Abrí el .txt en VS Code o en el Bloc de notas.
2. Andá a Archivo → Imprimir.
3. Como impresora, elegí Microsoft Print to PDF.
4. Guardalo como anexo/ia/javier_claude_01_diagnostico.pdf.

El PDF de Cristian tiene encabezados del tipo "Conversación con Claude — Cristian Morales — página X", así que seguramente usó otra herramienta. Si querés que los anexos queden con el mismo formato, preguntale cómo lo generó.

> **Usuario (Javier Ovelar) 24/10/2026**
>
> se puede guardar directamente a pdf o primero tiener que ser a texto?

Claude 24/10/2026 18:44

No, /export no guarda en PDF: siempre genera un archivo de texto. Así que primero tiene que ser texto y después lo convertís, por ejemplo con el "Imprimir → Microsoft Print to PDF" que te pasé.

Otra opción es que, cuando tengas el .txt exportado, yo te lo pase a PDF con un script, si en tu entorno hay alguna librería que sirva para eso; si no, habría que instalar una. Incluso le podría poner el mismo formato de encabezado que el de Cristian ("Conversación con Claude — Javier Ovelar — página X"), para que los anexos queden parejos.